Proofs as Processes, Continued

Cost Accounting, Reflection, and a Scalable Notion of Reasoning

L. G. Meredith*

May 2026

Abstract

Samson Abramsky's *Proofs-as-Processes* programme proposed that the proofs of a logic be realised as the processes of a concurrent calculus, with cut elimination as communication and linear logic as the logic of resources. In its mature, session-typed form the correspondence is tight — and tight in a way that confines it: it concerns the *normalisation* of an already-found proof, it is pinned to a fixed logic, and it secures its guarantees by statically forbidding the nonconfluence, deadlock, and divergence that proof *search* requires. We argue that four recent constructions on the rho calculus and on the category of continued interactive graph-structured lambda theories (ciGSLT) flesh the programme out and scale it. First, a cost-accounted rho calculus turns Abramsky's resources into a literal economy: tokens are linear resources consumed exactly once, the funding obligation is a judgement of a linear resource logic, and the validator that accepts a deployment is a proof checker performing linear proof search. Second, a cost endofunctor $\mathcal{C}$ on ciGSLT lifts this off any single calculus; the OSLF construction auto-generates a graded Hennessy–Milner logic from each theory, so the programme's "propositions" become modal formulae over a labelled transition system rather than formulae of one fixed logic, and an internalisation adjunction shows that over Turing-complete bases the apparatus is already definable. Third, reflection — the quote/dequote pair of the rho calculus — identifies an unproven obligation with a pending communication, places the search orchestrator *inside* the proof dynamics, and turns normalisation into decentralised, coordinator-free search. Fourth, the cost layer controls that search in two registers that are one ledger: an economic register of priced rules and budgets, and a biological register in which mortality (deadlock-by-starvation) prunes and metabolism (the splitter/joiner economy) reuses, with tactics evolving under selection in the manner of Buss. Proof identity becomes bisimulation, and completeness is recovered in the limit of fair scheduling and unbounded budget. The result is a notion of reasoning that scales where the original did not — parametric over logics, unifying checking and search, decentralised, resource-bounded with graceful degradation, and self-improving — and, fittingly, hosted on the chemical-abstract-machine substrate Abramsky reached for at the outset.

1 Introduction

The Curry–Howard correspondence ties the proofs of intuitionistic logic to the programs of the typed lambda calculus and proof normalisation to β-reduction. Abramsky's *Proofs-as-Processes* programme set out to extend that tie one dimension outward, from sequential functional computation to *concurrency* [1, 2]. Its proximate stimulus was Girard's observation that the multiplicative connectives of linear logic [4] describe communication without synchronisation problems,

*F1R3FLY.io; F1R3FLY Industries. lgreg.meredith@f1r3fly.io

together with the reading of cut elimination as parallel communication between processes; its goal was to exhibit a process calculus — Milner's π-calculus [7] was the candidate — as the computational correlate of a proof system, with linear logic supplying, it was hoped, a canonical and firm foundation for concurrency in the way Curry–Howard had for functional programming. Bellin and Scott made the paradigm precise, translating classical linear logic proof structures into a synchronous π-calculus and mirroring the proof-normalisation strategies as process reductions, with soundness and completeness [3]. Two decades later Caires and Pfenning, and then Wadler, tightened the correspondence into a session-typed discipline: linear propositions as session types, proofs as processes, cut elimination as communication, deadlock-freedom as a theorem [5, 6].

It is a beautiful programme, and the present paper is written in its debt. But read with an eye to *reasoning at scale* — not the verification of a fixed proof but the open-ended, distributed, resource-bounded search for proofs that real mathematical and verification practice demands — the programme as it stands has three structural boundaries, each of which is a virtue locally and a limitation globally.

(i) **It is pinned to a fixed logic.** The propositions are formulae of linear logic; the calculus is a session-typed fragment chosen to match them. A theory of reasoning should not have to be re-derived each time the logic changes.

(ii) **It concerns normalisation, not search.** Cut elimination runs a proof that already exists; it does not look for one. The genuinely hard and genuinely scalable activity — which rule, which lemma, which instantiation, explored in parallel across many candidates — lives outside the correspondence.

(iii) **Its tight form secures its guarantees by prohibition.** The session-typed calculi are confluent, strongly normalising, and deadlock-free *by typing*. These are exactly the three properties that proof search must give up: search is nonconfluent (the order of choices is its content), it uses deadlock as a pruning mechanism, and it must be permitted to diverge because the set of proofs is recursively enumerable but not recursive.

This paper argues that a series of four recent constructions [15, 16, 17, 18], developed on the rho calculus [13] and on the category ciGSLT of continued interactive graph-structured lambda theories, flesh out the programme along precisely these three axes, and that the combined effect is a notion of reasoning that scales where the original did not. The argument is not that Abramsky's correspondence was incomplete as mathematics — it is a theorem — but that the *ambition* it announced, linear logic as a firm foundation for concurrent reasoning, is realised more fully when three further moves are made: make the resources a runtime economy, lift the construction off any single logic, and put the search orchestrator inside the dynamics by reflection.

A historical remark frames the whole. Abramsky's interpretation of *classical* linear logic was given an operational semantics in the style of Berry and Boudol's Chemical Abstract Machine [9]: proofs as molecules in a solution, cut elimination as reaction. The proof-search construction we arrive at here [18] is, on the nose, a chemical abstract machine — goals, rules, lemmas and axioms floating in a soup, reacting when a redex is enabled. The substrate this paper proposes is, in a real sense, the one the programme reached for at its origin, now equipped with the three things the soup was missing: a way to price and bound its reactions, a way to leave the fixed logic, and a way to carry its own recipes as data.

Plan. Section 2 states the programme and its boundary precisely. Section 3 makes Abramsky's resources literal: the cost-accounted metered cut and the linear funding proof ([15]). Section 4 lifts the construction off the fixed logic: the cost endofunctor and OSLF-generated graded modal propositions ([16]). Section 5 turns normalisation into search: obligation as communication and the internalised orchestrator ([18]). Section 6 controls and evolves the search: economy and biology as one ledger ([17, 18]). Section 7 states what "scalable" means here, axis by axis, and records the formal obligations the programme now carries. Section 8 concludes.

2 The programme and its boundary

We fix vocabulary. A *cut* brings two proofs of dual formulae into contact and eliminates the shared formula; *cut elimination* is the rewriting that discharges cuts to reach a normal (cut-free) proof. The proofs-as-processes reading interprets a proof as a process, a formula as a communication protocol, and a cut as a channel along which two processes interact, so that one step of cut elimination is one act of communication [1, 3]. In the session-typed refinement the protocol is a session type, and the type discipline guarantees that well-typed processes never deadlock and always terminate [5, 6].

The decisive feature for our purposes is what the correspondence is *about*. It is about the dynamics of a proof that has already been found. A cut-free normal form is the goal; cut elimination transports a given proof to it. There is, in the relation itself, no room for the two affordances on which search depends. Confluence removes genuine choice: wherever cut elimination starts it ends at the same normal form, so the branching that search explores cannot be expressed in the reduction relation and must be supplied by a metalanguage outside it. Closedness removes interaction with open ends: a proof term, like a lambda term, has no free obligations reaching out for what they lack, so a partial proof is representable only as a term with a hole that some external agent inspects and fills. In the proof assistants that descend from this tradition — where a proof object is a term of a typed lambda calculus and checking is normalisation — the consequence is structural and well known: the search lives in a separate orchestration layer (an elaborator, a tactic engine, a scheduler) built in a different model of computation and bolted on top. The orchestrator sits beside the proof dynamics, never within it. This is not an engineering accident; it is forced by confluence and closedness [18].

The session-typed tightening sharpens the point into an inversion. Well-typed CP is deadlock-free, confluent, and strongly normalising, and what would be choice is typed into race-free internal and external choice [6]. These are the properties that make cut elimination behave like a finished computation — and they are exactly the three that proof search is built to exercise: nonconfluent choice (the content of search is which choice, in which order), deadlock (a failed branch that cannot pay is pruned by getting stuck), and potential divergence (soundness-and-completeness over a recursively enumerable but non-recursive set of proofs forbids guaranteed termination). A discipline that forbids all three statically, by typing, is the wrong control surface for search; the right one keeps all three live and disciplines them dynamically. That observation is the seam along which the following sections open the programme.

3 Resource made literal: the metered cut

Linear logic is the logic of resources: a linear hypothesis is consumed exactly once. Abramsky's programme inherits this as a property of proofs. The first construction [15] inherits it as a

property of *running processes*, by folding a resource economy directly into the grammar and operational semantics of the rho calculus rather than leaving it to the type discipline.

3.1 Tokens as linear hypotheses

The cost-accounted rho calculus signs terms with cryptographic signatures and introduces a first-class *token stack* $S ::= () \mid s{:}S$, a list of fuel cells each keyed to a signature s. Communication is gated: the communication rule of the pure calculus is replaced by a family of rules in which a redex fires only when a matching token is present, consuming it. In its simplest single-signature form,

$$\{\,\mathsf{for}(y \leftarrow x)\,T \;\mid\; x!(U)\,\}_s \;\mid\; s{:}S \;\longrightarrow\; T\{@U/y\} \;\mid\; S.$$

A token is a linear resource: present or absent, consumed exactly once, never duplicated by the rule. The five rules that govern signed communication are not five facts but the single old rule of communication seen through the requirement that someone pay [15]. The *linear* of linear logic has become operational: not a discipline on hypotheses in a derivation but a discipline on tokens in a tuplespace.

3.2 The funding obligation is a linear proof

The construction's transactional analysis makes the linear-logic connection explicit rather than allusive. Define, for a signed deployment and a signature s, the *demand* Δ_s as the number of s-gated layers reachable in its call graph, and the *supply* Σ_s as the depth of s-keyed tokens available. A deployment is accepted exactly when, for every signature appearing in it,

$$\Sigma_s \;\geq\; \Delta_s,$$

a linear resource inequality: each token is consumed once, and supply must meet demand [15]. This is not an analogy. The funding obligation is a judgement in the resource-sensitive fragment of linear logic; the demand counts the linear hypotheses required, the supply counts those available; a successful funding check is a proof in a linear resource logic that the deployment is well-resourced. Two deployments competing for the same tokens are two proof obligations competing for the same linear hypotheses, and at most one can succeed — not by a scheduling heuristic but as a consequence of linearity. The validator that admits a deployment is, in the precise sense, a *proof checker*, and acceptance is *linear proof search*. The lollipop sugar of the calculus, $\{\cdots\}_{s_1 \multimap s_2}$, in which one signer funds the communication and another the continuation, is named for the linear implication it instances [15].

The seed of the whole programme-completion is already here. Abramsky's resource logic, given a runtime and an economy, is not merely interpreted by processes; it *runs as* a proof-search engine in which the accepting authority is a checker. The later sections generalise the logic, internalise the orchestrator, and control the search; but the identification "acceptance is linear proof search" is the first instance of the thesis.

3.3 The symmetric metered cut

To prepare the lift off the rho calculus, the second construction abstracts the shape of the gated rule [16]. Write a generating dynamics as a *symmetric metered cut*: an interaction constructor

K brings two (co-)introductions into contact, and a partial contraction $\mathsf{compute}$ transforms their continuations,

$$(C'_p, C'_e) = \mathsf{compute}(I, J, C_p, C_e) \qquad K\big(K_p(I, C_p),\, K_e(J, C_e)\big) \;\longrightarrow\; K'(C'_p, C'_e). \qquad (\dagger)$$

In the rho calculus $K = |$, the receiver $K_p = \mathsf{for}$ carries continuation T, the sender $K_e = !$ carries payload U with the same status as T, and $\mathsf{compute}$ is substitution. In the lambda calculus $K = \mathsf{App}$, $K_p = \lambda$, the argument is its own (degenerate) co-introduction, and $\mathsf{compute}$ is again substitution. The cut is *symmetric*: the eliminand is not a bare term but a co-introduction with its own continuation — which is exactly the situation in proofs-as-processes, where the two cut formulae are dual and neither party is privileged. The metered version interposes a token between the operands of K and drains it on contact. "Meter the cut" is the generic content of "price every interaction."

4 Off the fixed logic: the cost endofunctor

Abramsky's correspondence chose a logic (linear logic) and built a matching calculus. The second construction [16] removes the choice by making the cost construction a functor on a category of theories, and by generating the logic from each theory rather than fixing it in advance.

4.1 A category of theories

A *graph-structured lambda theory* (GSLT) is a multi-sorted term theory $G = (\Sigma, E, R)$ — a signature of sorts and operators, equations imposing a structural congruence, and rewrite rules. It is *interactive* when a distinguished sort carries a labelled transition system on which bisimulation is the intended equivalence; the rho calculus and the lambda calculus are the motivating objects of the category iGSLT [16]. A *continued interactive* GSLT additionally presents its dynamics as a symmetric metered cut (†), comes equipped with a computable canonical-form function (a section of its structural-congruence quotient), and has a contraction that preserves wrapping; these objects form the category ciGSLT, a proper structural enrichment of iGSLT.

Construction 4.1 (The cost endofunctor, [16])**.** There is an endofunctor $\mathcal{C} : \mathsf{ciGSLT} \to \mathsf{ciGSLT}$ that, on an object G, adjoins a signature sort, a token-stack sort, and a wrapped-term sort; makes the contact constructor K polymorphic over wrapped terms and stacks; re-sorts every continuation slot so that each redex is born inside a token-gated wrapper; and replaces the dynamics by the gated family of rules. The morphisms it acts on are the bisimulation-preserving, quote-faithful theory maps.

The discipline of the image is grammatical: every continuation is a metered thunk $\{P\}_s$, so "no redex fires without consuming a token" is a sorting invariant preserved by reduction, not a property re-established by traversal. Applying the rho instance recovers the five-rule lattice of Section 3; applying the lambda instance meters β-reduction. The construction needs not a channel constructor but a section: for lambda the section is the de Bruijn encoding, and a signature commits to the de Bruijn normal form [16]. Abramsky's programme worked the lambda/process distinction one calculus at a time; here the cost construction is uniform across the category, with the lambda calculus as a control showing that what is required is canonicalisation, not communication.

4.2 The logic is generated, and graded

What plays the role of Abramsky's fixed propositions is, in this setting, generated. The OSLF construction takes a GSLT with an adequacy theorem and auto-generates a Hennessy–Milner modal logic over its transition system, adequate for bisimulation [8]. Applied to the graded transition system of $\mathcal{C}(G)$ — whose steps are labelled by the signatures they consume — OSLF yields a *graded* Hennessy–Milner logic with modalities $\langle a \rangle_s$ indexed by consumed signature.

Theorem 4.2 (Graded adequacy, schematic [16])**.** *For $\mathcal{C}(G)$ the graded Hennessy–Milner logic generated by OSLF is adequate for quote-faithful bisimulation: two configurations satisfy the same graded formulae iff they are quote-faithfully bisimilar.*

Two consequences matter for the programme. First, the "propositions" of proofs-as-processes are no longer formulae of one chosen logic; they are modal formulae over whatever theory one reasons in, manufactured from it. The correspondence becomes parametric: *any* continued interactive theory, *its own* generated logic. Second, linear logic does not disappear in the generalisation — it reappears twice. It is the meta-logic of the funding obligation (Section 3); and it grades the object logic, since the modal indices s are the consumed resources, so resource-sensitivity is carried in the modalities. Abramsky's "linear logic as resource logic" is thus deepened: resource is both the control layer and a grading on the propositions.

4.3 The apparatus is, over universal bases, already there

Iterating the construction flattens — signatures of signed terms multiply, stacks of stacks concatenate — so $\mathcal{C}$ carries a monad structure, a *non-idempotent* resource monad: re-metering a metered calculus yields a finer account that the multiplication consolidates [16]. The monad resolves through two adjunctions of opposite character. The first, install $\dashv$ forget, is structural: the apparatus is a detachable layer, installed freely and forgotten on the nose — structure-preserving but behaviour-altering, since forgetting the tokens removes the gating. The second holds only over a Turing-complete base: there the entire metering apparatus is computable, hence encodable in the base itself, and $\mathcal{C}(G)$ is a behavioural retract of G up to weak bisimulation — cost accounting is a *definitional extension*, behaviour-preserving but structure-dissolving.

Remark 4.3. *The second adjunction is the formal content of "the apparatus was already there." Over a universal substrate, pricing interaction adds no power the substrate lacked; it names a structure the substrate already contained. This both vindicates Abramsky's hope — that a sufficiently expressive process calculus is the right home for a proof system — and sharpens it: the cost layer that controls reasoning is not foreign to the universal calculus but internal to it. Which representation that internalisation takes, however, is not fixed by universality, and the next section turns on exactly that distinction.*

5 From checking to searching: obligation as communication

The move from normalisation to search is the heart of the matter, and it turns on a single identification and a single capability [18].

5.1 The identification

Definition 5.1 (Obligation as communication [18])**.** An unproven obligation is a pending communication. An open goal is a process waiting to receive on a channel typed by the goal; a

lemma, axiom, or rule instance is a process able to send; a completed proof is a configuration with no pending obligations and no enabled interaction — a normal form in the behavioural sense.

In the session-typed base case this is already literal: a free channel typed by a formula is an open hypothesis. Generalising the carrier from the session-typed π-calculus to an arbitrary continued interactive GSLT, the open goals of a partial proof become the free names and pending inputs of a proof-process, and *checking and search cease to be two activities at two levels.* They are one dynamics observed at different stages of saturation: search is the dynamics of partial, open proof-processes seeking interaction partners; checking is the same dynamics once no obligation remains pending.

The disjunctive and conjunctive structure of search realises natively the two affordances the lambda calculus lacked. An OR-node — the choice of which rule to try — is competing communication on a shared channel, the genuine nondeterminism confluence had removed. An AND-node — the several premises a rule demands — is a join pattern,

$$\mathsf{for}\big(\, y_1 \leftarrow a_1 \,\&\, \cdots \,\&\, y_n \leftarrow a_n \,\big)\, P,$$

which fires only when all n inputs are simultaneously available: no coordinator assembles the conjunction, the join does. The reaction is the structural-congruence-plus-communication semantics of the calculus, read as a chemical abstract machine [9, 10] — which, as noted in the introduction, is the operational setting Abramsky's classical interpretation already used. There are honest precedents for proof search as soup dynamics: concurrent constraint programming with ask/tell on a shared store [12], and the uniform-proofs tradition that makes search the operational semantics of a logic [11]. The deltas here are reflection and an internalised cost layer.

5.2 The capability: reflection puts the orchestrator inside

The lambda calculus and the session calculi are first-order in the relevant sense: channels carry channels, not proof terms one can run as data. To obtain proofs-as-data one bolts on a higher-order layer or a metalanguage — and reintroduces the object/meta split. The rho calculus supplies the layer inside one calculus: quotation $@P$ turns a process into a name, dereference $*x$ turns a name back into the process it quotes [13]. A proof-process can therefore reify its own open obligations as data, route them, inspect them, transform them, and run them. "Manipulate an obligation" and "manipulate a process as data" become one operation, and the orchestrator — the search strategy — lives in the same calculus as the proofs. The session-typed base case did not need reflection because normalisation does not; search does [18].

5.3 Why rho, and not CP

The inversion of Section 2 now pays off. CP disciplines away nonconfluence, deadlock, and divergence statically, by typing, as prohibitions [6]. The rho calculus keeps all three live and disciplines them dynamically, by an economy: nonconfluent choice is competing communication, deadlock-by-starvation is the pruning mechanism, and divergence is permitted and metered rather than forbidden. For a search substrate the pathologies on which search depends are precisely those a type discipline exists to forbid; the choice is not of a worse-behaved calculus but of the one whose control is in the right place. Moreover, moving proof dynamics into a nonconfluent model costs

the canonicity of normal forms, so proof identity can no longer be $\beta\eta$-equality. The right notion is bisimulation — and on the rho calculus this is not an imposition but the home equivalence.

Proposition 5.2 (Proof identity is bisimulation [18])**.** *Two proof-processes denote the same proof iff they are weakly bisimilar, equivalently iff no generated Hennessy–Milner formula separates them.*

So a property that would be a redefinition in CP is, on rho, the default reading, and it lands directly on the modal logic of Section 4: "same behaviour" and "same proof" coincide, characterised by the auto-generated logic. The cost incurred on a generic substrate is free on rho specifically.

6 Controlling and evolving search: one ledger

A substrate that can search must also be told how hard to look, where to stop, and how to improve. The cost layer that made the calculus mortal is exactly the control surface, and it controls search in two registers that are one ledger [17, 18].

6.1 The economic register

Price each inference rule or tactic in phlogiston: a rewrite cheap, a decision procedure dear. A goal-process carries a budget and can fire only the rules it can afford, so iterative deepening, best-first, and breadth-versus-depth become budget policies rather than coordinator algorithms. Because the meter lives in the same calculus as the proof, a goal-process can reason about its own budget and reallocate — self-budgeting search, a meta-level strategy with no meta-level interpreter.

6.2 The biological register

The third construction [17] reads the same cost layer as a biology, and the reading transfers intact to search. *Death* is deadlock-by-starvation: a branch whose token stack is exhausted has no enabled move and blocks permanently. Read as search, an unproductive branch prunes itself — no coordinator decides to kill it; it fails to pay and deadlocks. That is the decentralisation claim made mechanical. *Metabolism* supplies reuse: a proven lemma is a high-energy metabolite that downstream goals consume, and the splitters and joiners that reconcile token granularity are the catabolism and anabolism of proof-effort, reallocating fuel toward branches that produce reusable structure. *Heredity* supplies improvement: reflection picks out a genome — a quoted description $@P$ copied uninterpreted (replication) and dequoted to run (expression) — so a tactic that closes goals cheaply can replicate into fresh contexts, with the phlogiston economy as the selection pressure. The variation operator is the framework's own ability to mutate and recombine the rewrite rules of a defined proof-language: its types, equations, and rewrites play the role of a genome, and the cost economy does the selecting.

This last point connects the programme to Buss's account of why mortal individuals exist at all [14]. In an unmetered calculus an immortal replicator with a faster cycle invades any parallel composition without bound; no higher-level whole is stable against its own fastest part. Metering caps the invasion: a high-draw defector runs only until its endowment is spent, then dies [17]. Somatic mortality together with germ sequestration aligns the levels of selection by denying a within-individual defector any future — which, transposed to search, is the statement

that a runaway unproductive tactic cannot monopolise the soup, because it cannot pay forever. Selection favours mortal searchers for the same reason it favours mortal organisms.

6.3 One economy

The payoff of doing the work on the rho calculus is that these are not three control systems but one. The gas that meters a blockchain node, the budget that steers a proof search, and the energy that drives a metabolism are one phlogiston economy on one substrate, the same coin under three readings: the node's fuel *is* a search budget *is* a metabolism [18]. An ecosystem of cost-accounted reflective processes, contending over a bounded fuel supply, selecting variants by who can pay, is — with no addition and no analogy — a decentralised proof search with no overarching coordinator, in which mortality prunes, metabolism reuses, and reproduction explores.

7 A scalable notion of reasoning

We can now say precisely what "scalable" means and against what baseline. The baseline is the programme as Abramsky launched it and the session calculi matured it: a single-threaded cut elimination running an already-found proof of a fixed logic, with search, when needed, supplied by an external orchestrator. The constructions scale this along five axes.

S1. Across logics. Fixed linear logic → any continued interactive GSLT, with propositions the OSLF-generated graded Hennessy–Milner logic of that theory (Section 4). The correspondence is parametric, not pinned.

S2. Across the checking/search divide. Normalisation of a found proof → search for a proof, unified as one dynamics at different saturations (Section 5). Checking is search with no obligations left.

S3. Across agents. A single reduction sequence with an external coordinator → many concurrent goal- and rule-processes, coordinator-free, with AND-nodes as join patterns and OR-nodes as competing communications (Section 5).

S4. Across resource bounds. Typed-to-terminate → budget-controlled and anytime, with completeness degrading gracefully: under fair scheduling and unbounded budget the reachable closed configurations are exactly the recursively enumerable set of proofs, and every finite budget restricts that set monotonically, one starved branch at a time (Section 6).

S5. Across time. A fixed tactic set → tactics as an evolvable genome, mutated and recombined under phlogiston selection, with mortality pruning and metabolism reusing (Section 6).

The unifying fact beneath all five is that one phlogiston economy, on one reflective concurrent substrate, simultaneously realises the resource logic of funding, the modal logic of the propositions, the budget policy of the search, and the metabolism of the population — with proof identity given by bisimulation throughout. Table 1 summarises the contrast.

	Proofs-as-Processes (classical)	**Continued (this synthesis)**
Logic	fixed (linear)	generated per theory (graded HML)
Activity	normalisation	search (checking as its limit)
Choice	confluent / typed away	nonconfluent competing communication
Deadlock	forbidden by typing	pruning by starvation
Termination	guaranteed	metered; complete in the limit
Orchestrator	external metalanguage	internal, via reflection
Control	none (or types)	one phlogiston economy
Improvement	none	tactic evolution under selection
Identity	$\beta\eta$ / typed	bisimulation

Table 1: What the four constructions change.

The formal obligations. The synthesis is, at this stage, a programme with three theorem-shaped targets carried over from [18], each of which the foregoing makes precise enough to attempt.

(1) *Encoding-adequacy of decentralised search.* A search strategy specified as a global AND/OR traversal is realised by a parallel composition of goal- and rule-processes such that the reachable terminal configurations correspond, up to bisimulation, to the proofs the strategy finds, with no process holding global search state.

(2) *Fairness-and-budget completeness.* Under a fairness condition on the transition system and an unbounded budget, the concurrent search is complete: the reachable closed configurations coincide with the recursively enumerable set of proofs, and finite budgets restrict this set monotonically.

(3) *Proof identity is bisimulation.* Two proof-processes denote the same proof iff they are weakly bisimilar, the characterisation factoring through the generated modal logic so that "same behaviour" and "same proof" provably coincide.

These are the same three obligations the resource, functorial, and reflective constructions impose when read as a theory of reasoning; that the logical targets and the ecological ones (a well-defined sublanguage of mortal processes, evolvability as a property of the description algebra, the logistic law as a genuine hydrodynamic limit [17]) turn out to be the same targets is not a coincidence to be explained away. It is the content of the claim that one substrate carries both.

Remark 7.1 (What cost accounting does and does not buy)**.** *Cost accounting never converts an undecidable search into a decidable one. What it converts is the character of nontermination: from "search may silently diverge" to "search consumes a controllable budget at a controllable rate" [18]. This is the anytime, resource-bounded reframing one wants in practice, and the right place to host the undecidability that Abramsky's normalisation-only correspondence never had to confront, because it never searched.*

8 Conclusion

Abramsky framed computation as interaction and gave proofs a process reading, in which cut elimination is communication and linear logic is the logic of resources. The programme's reach was bounded in three ways that its later, tight refinements made sharper rather than looser: it

ran an already-found proof, in a fixed logic, with the phenomena search needs disciplined away. The four constructions synthesised here remove the three bounds in turn. The cost-accounted rho calculus makes the resources a literal economy and the accepting authority a proof checker performing linear proof search. The cost endofunctor lifts the construction off any single logic and generates the propositions, as a graded modal logic, from whatever theory one reasons in, while showing that over universal bases the apparatus is already definable. Reflection identifies an open obligation with a pending communication and places the orchestrator inside the dynamics, turning normalisation into decentralised search. And the cost layer controls that search in two registers — economic and biological — that are one ledger, with tactics evolving under selection and proof identity given by bisimulation.

The substrate that results is the chemical abstract machine Abramsky's classical interpretation reached for at the start, now able to price and bound its reactions, to leave the fixed logic, and to carry its own recipes as data. On it, reasoning is concurrent reaction: many partial proofs interacting in a soup, paying as they go, the unproductive starving, the useful replicating, the whole under a single economy that is at once a gas, a budget, and a metabolism. That is a more scalable notion of reasoning than a single thread eliminating cuts in a fixed logic — and it is, we have argued, the programme's own ambition, carried to where its substrate could always have taken it.

E pluribus, ratio.

References

[1] S. Abramsky. Computational interpretations of linear logic. *Theoretical Computer Science* 111(1–2):3–57, 1993.

[2] S. Abramsky. Proofs as processes. *Theoretical Computer Science* 135(1):5–9, 1994.

[3] G. Bellin and P. J. Scott. On the π-calculus and linear logic. *Theoretical Computer Science* 135(1):11–65, 1994.

[4] J.-Y. Girard. Linear logic. *Theoretical Computer Science* 50(1):1–101, 1987.

[5] L. Caires and F. Pfenning. Session types as intuitionistic linear propositions. In *CONCUR 2010*, LNCS 6269, pp. 222–236. Springer, 2010.

[6] P. Wadler. Propositions as sessions. *Journal of Functional Programming* 24(2–3):384–418, 2014. (Conference version, ICFP 2012.)

[7] R. Milner, J. Parrow, and D. Walker. A calculus of mobile processes, I. *Information and Computation* 100(1):1–40, 1992.

[8] M. Hennessy and R. Milner. Algebraic laws for nondeterminism and concurrency. *Journal of the ACM* 32(1):137–161, 1985.

[9] G. Berry and G. Boudol. The chemical abstract machine. *Theoretical Computer Science* 96(1):217–248, 1992.

[10] J. Meseguer. Conditional rewriting logic as a unified model of concurrency. *Theoretical Computer Science* 96(1):73–155, 1992.

[11] D. Miller, G. Nadathur, F. Pfenning, and A. Scedrov. Uniform proofs as a foundation for logic programming. *Annals of Pure and Applied Logic* 51(1–2):125–157, 1991.

[12] V. A. Saraswat, M. Rinard, and P. Panangaden. Semantic foundations of concurrent constraint programming. In *POPL 1991*, pp. 333–352. ACM, 1991.

[13] L. G. Meredith and M. Radestock. A reflective higher-order calculus. *Electronic Notes in Theoretical Computer Science* 141(5):49–67, 2005.

[14] L. W. Buss. *The Evolution of Individuality*. Princeton University Press, 1987.

[15] L. G. Meredith. Cost-accounted rho calculus: a spectral decomposition of phlogiston. F1R3FLY.io technical note, 2026.

[16] L. G. Meredith. Continued interactive GSLTs and the cost endofunctor. F1R3FLY.io technical note, 2026.

[17] L. G. Meredith. Mortal computation: mortality, the genome, and life-history in cost-accounted GSLTs. F1R3FLY.io technical note, 2026.

[18] L. G. Meredith. Proof search as concurrent reaction: the rho calculus and the category of continued interactive GSLTs as a substrate for proof objects with internalised, cost-accounted search. F1R3FLY.io technical note, 2026.